

Une Approche Hybride ML–Métaheuristique : Algorithme Génétique et VNS pour l'Optimisation du Flow Shop

HAMROUN Mohammed Ali
Département Informatique
Alger, Algérie
mm_hamroun@esi.dz

LADOUL Mehdi
Département Informatique
Alger, Algérie
mm_ladoul@esi.dz

SALHI Fatma
Département Informatique
Alger, Algérie
mf_salhi@esi.dz

SADOUN Amel
Département Informatique
Alger, Algérie
ma_sadoun@esi.dz

BENAHMED Lyna Kamilia
Département Informatique
Alger, Algérie
ml_benahmed@esi.dz mm_chekman@esi.dz

CHEKMAN Meysssem
Département Informatique
Alger, Algérie

Superviseurs: ENS KECHID Amine, ENS BESSEDIK Malika
École nationale Supérieure d'Informatique, Alger, Algérie
Mai 2026

I. INTRODUCTION

II. FORMULATION MATHÉMATIQUE DU PROBLÈME

A. Définition formelle du problème

Le problème traité dans ce travail est le Permutation Flow Shop Problem (PFSP), une variante du Flow Shop Scheduling (FSP) dans laquelle l'ordre de passage des jobs sur les machines est identique pour toutes les machines. Ce problème modélise un environnement de production dans lequel un ensemble de n jobs doit être exécuté séquentiellement sur m machines, chaque job devant obligatoirement passer par les machines dans le même ordre fixé.

Le PFSP appartient à la classe des problèmes NP-difficiles dès que $m \geq 3$ [1], ce qui exclut toute résolution exacte efficace pour des instances de taille réaliste. Dans ce contexte industriel, minimiser le makespan — soit le temps d'achèvement du dernier job sur la dernière machine — constitue l'objectif central, car il reflète directement le rendement global du système de production.

B. Formulation mathématique

Paramètres et indices :

- n : nombre de jobs, indexés par $i \in \{1, \dots, n\}$.
- m : nombre de machines, indexées par $k \in \{1, \dots, m\}$.
- $p(i, k)$: temps de traitement du job i sur la machine k , supposé entier, non-préemptif et déterministe.

Variable de décision : Une solution est représentée par une permutation $\pi = (\pi_1, \pi_2, \dots, \pi_n)$ des n jobs, où π_j désigne le job placé à la j -ème position dans la séquence. L'espace de recherche est donc l'ensemble de toutes les permutations possibles, de cardinalité $n!$.

Calcul du makespan : Les temps de complétion $C(\pi_j, k)$ sont calculés récursivement selon les contraintes de précedence et de disponibilité machine :

$$C(\pi_1, 1) = p(\pi_1, 1), \quad (1)$$

$$C(\pi_1, k) = C(\pi_1, k-1) + p(\pi_1, k), \quad \forall k \in \{2, \dots, m\}, \quad (2)$$

$$C(\pi_j, 1) = C(\pi_{j-1}, 1) + p(\pi_j, 1), \quad \forall j \in \{2, \dots, n\}, \quad (3)$$

$$C(\pi_j, k) = \max\{C(\pi_{j-1}, k), C(\pi_j, k-1)\} + p(\pi_j, k), \quad \forall j \geq 2, k \geq 2. \quad (4)$$

Fonction objectif : L'objectif est de trouver la permutation π^* qui minimise le makespan :

$$\pi^* = \arg \min_{\pi} C_{\max}(\pi) = \arg \min_{\pi} C(\pi_n, m). \quad (5)$$

Contraintes :

- Chaque job ne peut être traité que sur une machine à la fois (contrainte de non-préemption).
- Chaque machine ne peut traiter qu'un seul job à la fois (contrainte d'exclusivité machine).
- L'ordre de traitement est identique sur toutes les machines (contrainte de permutation).
- Les temps de traitement sont supposés connus, déterministes et strictement positifs.

La combinaison de ces contraintes et de la croissance factorielle de l'espace de recherche justifie pleinement le recours à des approches métaheuristiques — ici un Algorithme Génétique (AG) et une Recherche à Voisinage Variable (VNS) — renforcées par un agent Q-Learning pour le pilotage adaptatif des paramètres, tel que décrit dans la section suivante.

III. PARTIE I : APPROCHE HYBRIDE RL-GA

L'approche proposée, désignée RL-GA, repose sur l'intégration d'un agent d'apprentissage par renforcement (Reinforcement Learning) au sein d'un algorithme génétique (AG) afin de contrôler dynamiquement ses paramètres de sélection et de mutation. Contrairement à un AG classique dont les paramètres sont fixés statiquement avant l'exécution, l'agent RL observe à chaque génération l'état courant de la population et prend des décisions adaptatives visant à maximiser la qualité des solutions produites.

L'hybridation repose sur une interaction cyclique entre l'agent RL et l'AG : à chaque génération, l'agent reçoit une représentation de l'état de la population, sélectionne une action parmi 27 combinaisons de paramètres, applique cette action aux opérateurs génétiques, puis reçoit un signal de récompense calculé sur la base de l'amélioration du makespan observée. La Q-Table est mise à jour en conséquence selon l'équation de Bellman, permettant à l'agent d'affiner progressivement sa politique de contrôle au fil des générations. La Figure 1 illustre ce schéma d'interaction.

A. Architecture globale

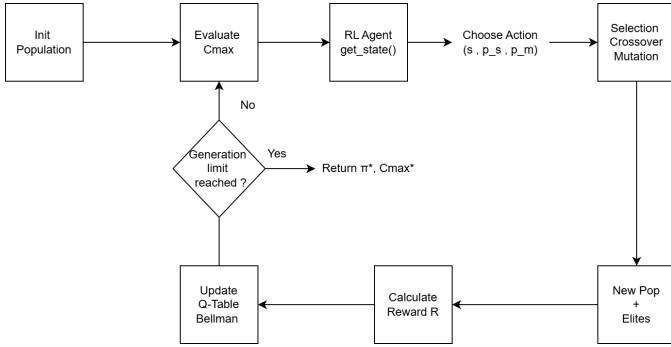


Fig. 1. Architecture globale de l'approche hybride RL-GA pour le FSP.

B. Codage et représentation

L'approche RL-GA repose sur deux niveaux de représentation complémentaires : la représentation des solutions du PFSP au sein de l'AG, et la représentation de l'état de la population pour l'agent RL.

1) *Représentation d'un Individu*: Chaque individu est encodé comme une permutation $\pi = (\pi_1, \pi_2, \dots, \pi_n)$ des n jobs, stockée sous forme de liste d'entiers. Cette représentation est directement compatible avec les opérateurs génétiques retenus, qui garantissent la validité de la permutation après chaque opération sans mécanisme de réparation.

2) *Représentation de l'État pour l'Agent RL*: L'état S_t transmis à l'agent à chaque génération t est encodé sous forme d'un entier :

$$S_t = \text{niv_f} \times 3 + \text{niv_H} \in \{0, \dots, 8\} \quad (6)$$

où $\text{niv_f} \in \{0, 1, 2\}$ est le niveau de fitness moyen normalisé et $\text{niv_H} \in \{0, 1, 2\}$ est le niveau d'entropie de la population, donnant $|S| = 9$ états distincts.

3) *Représentation d'une Action*: Une action $a_t = (s, p_s, p_m)$ est encodée comme un indice entier dans $\{0, \dots, 26\}$, correspondant à l'une des 27 combinaisons du produit cartésien :

$$\mathcal{A} = \{s\} \times \{p_s\} \times \{p_m\}, \quad |\mathcal{A}| = 27 \quad (7)$$

où chaque composante prend les valeurs suivantes :

TABLE I
COMPOSANTES DE L'ESPACE DES ACTIONS

Composante	Valeurs	Cardinalité
s (méthode)	{élitisme, roulette, rang}	3
p_s (taux sélection)	{0.3, 0.6, 0.9}	3
p_m (taux mutation)	{0.1, 0.3, 0.5}	3
Total		$3 \times 3 \times 3 = 27$

4) *Structure de la Q-Table*: La politique de l'agent est stockée dans une Q-Table $Q \in \mathbb{R}^{9 \times 27}$, initialisée à zéro. Chaque entrée $Q(s, a)$ représente la valeur estimée de choisir l'action a dans l'état s , mise à jour en temps réel à chaque génération selon l'équation de Bellman.

C. Composants ML : Agent Q-Learning

1) *Représentation de l'État*: À chaque génération t , l'agent observe deux indicateurs sur la population courante.

a) Fitness moyen normalisé :

$$\text{niv_f} = \min \left(\left\lfloor \frac{\bar{f} - f_{\min}}{f_{\max} - f_{\min}} \times 3 \right\rfloor, 2 \right) \quad (8)$$

b) Entropie de Shannon :

$$p_m = f(x_m)^{-1} / \sum_{m=1}^M f(x_m)^{-1} \quad (9)$$

$$H = - \sum_{m=1}^M p_m \cdot \log_2(p_m) \quad (10)$$

$$\text{niv_H} = \min \left(\left\lfloor \frac{H}{\log_2(M)} \times 3 \right\rfloor, 2 \right) \quad (11)$$

L'état est encodé sous forme d'un entier :

$$S_t = \text{niv_f} \times 3 + \text{niv_H} \in \{0, \dots, 8\} \quad (12)$$

La combinaison de niv_f et niv_H produit un espace d'états de cardinalité $|S| = 9$, résumé dans la Table II.

TABLE II
ESPACE DES 9 ÉTATS DE L'AGENT RL

	H=0 (div. faible)	H=1 (div. moyenne)	H=2 (div. forte)
F=0 (bonne)	état 0	état 1	état 2
F=1 (moyenne)	état 3	état 4	état 5
F=2 (mauvaise)	état 6	état 7	état 8

2) Espace des Actions:

Une action $a_t = (s, p_s, p_m)$ est choisie parmi 27 combinaisons possibles, définies par le produit cartésien :

$$\mathcal{A} = \{s\} \times \{p_s\} \times \{p_m\}, \quad |\mathcal{A}| = 27 \quad (13)$$

où $s \in \{\text{élitisme, roulette, rang}\}$ désigne la méthode de sélection des parents, $p_s \in \{0.3, 0.6, 0.9\}$ le taux de sélection et $p_m \in \{0.1, 0.3, 0.5\}$ le taux de mutation.

Le choix de l'action suit une politique ε -greedy équilibrant exploration et exploitation :

$$a_t = \begin{cases} \text{action aléatoire} & \text{si } \text{rand}() < \varepsilon \\ \arg \max_a Q(S_t, a) & \text{sinon} \end{cases} \quad (14)$$

Avec probabilité ε , l'agent explore en choisissant une action aléatoire afin de découvrir de meilleures combinaisons de paramètres. Avec probabilité $1 - \varepsilon$, il exploite la politique apprise en sélectionnant l'action de valeur Q maximale dans l'état courant S_t . Le paramètre ε est initialisé à 0.5 et décroît linéairement jusqu'à une valeur plancher de 0.05, favorisant progressivement l'exploitation au fil des générations.

3) Fonction de Récompense:

Après construction de la nouvelle génération, l'agent calcule la récompense R_t composée de deux termes complémentaires.

Récompense enfants évalue l'amélioration individuelle de chaque paire parent-enfant générée :

$$R_{children} = \sum_i [f(p_{1i}) + f(p_{2i}) - f(e_{1i}) - f(e_{2i})] \quad (15)$$

où $f(p_{1i}), f(p_{2i})$ désignent le Cmax des deux parents et $f(e_{1i}), f(e_{2i})$ celui des deux enfants issus du croisement. Un enfant meilleur que son parent contribue positivement à ce terme.

Récompense sélection évalue l'amélioration du meilleur individu entre deux générations consécutives :

$$R_{selection} = f(best_t) - f(best_{t+1}) \quad (16)$$

où $f(best_t)$ est le Cmax du meilleur individu à la génération t et $f(best_{t+1})$ celui de la génération suivante.

La récompense totale est alors :

$$R_t = R_{children} + R_{selection} \quad (17)$$

Une valeur $R_t > 0$ indique que l'action a_t a amélioré la population et sera favorisée lors des prochaines générations. Une valeur $R_t < 0$ pénalise l'action, réduisant sa probabilité d'être choisie dans un état similaire.

4) Mise à Jour de la Q-Table:

La mise à jour se fait à travers cette equation :

$$Q(S_t, a_t) \leftarrow (1 - \alpha) Q(S_t, a_t) + \alpha [R_t + \gamma \max_a Q(S_{t+1}, a)] \quad (18)$$

Cette équation combine trois termes :

- **Valeur actuelle** $(1 - \alpha) Q(S_t, a_t)$: la connaissance accumulée par l'agent sur l'action a_t dans l'état S_t ,

pondérée par $(1 - \alpha)$ pour contrôler l'amplitude de la mise à jour.

- **Récompense immédiate** R_t : le signal de retour calculé après application de l'action, défini par $R_t = R_{children} + R_{selection}$ (équations (8) et (9)). Une valeur positive indique que l'action a amélioré la population ; une valeur négative la pénalise.
- **Récompense future estimée** $\gamma \max_a Q(S_{t+1}, a)$: la meilleure valeur Q atteignable depuis l'état suivant S_{t+1} , actualisée par le facteur γ . Ce terme permet à l'agent d'anticiper les conséquences à long terme de ses décisions et non uniquement l'effet immédiat.

Les hyperparamètres $\alpha = 0.1$, $\gamma = 0.9$ et $\varepsilon_0 = 0.5$ ont été calibrés conformément aux recommandations de Irmouli et al.

5) Rôle de l'Agent et Conclusion:

À chaque génération, l'agent Q-Learning joue le rôle d'un contrôleur adaptatif au sein de l'AG. Son cycle d'interaction se résume en quatre étapes : il **observe** l'état S_t de la population via le fitness moyen et l'entropie, il **décide** de l'action $a_t = (s, p_s, p_m)$ selon la politique ε -greedy, il **évalue** la qualité de sa décision via la récompense R_t , puis il **met à jour** la valeur $Q(S_t, a_t)$ selon l'équation de Bellman.

Au fil des générations, la Q-Table accumule l'expérience de l'agent : les actions ayant conduit à une amélioration du Cmax voient leur valeur Q augmenter, tandis que les actions dégradant la population sont progressivement pénalisées. L'agent apprend ainsi automatiquement quelle combinaison de méthode de sélection et de taux de mutation est la plus adaptée à chaque situation de la population, sans intervention humaine.

IV. PARTIE II : APPROCHE HYBRIDE ML-VNS

En complément de l'approche RL-GA, nous proposons une seconde hybridation fondée sur la Recherche à Voisinage Variable (Variable Neighborhood Search, VNS) [4], [5]. Cette métaheuristique est particulièrement adaptée au PFSP car elle combine l'**intensification**, par une recherche locale autour d'une bonne permutation, et la **diversification**, par l'utilisation progressive de voisinages plus perturbateurs.

L'idée centrale de l'approche ML-VNS est de remplacer le choix fixe du voisinage par un mécanisme d'apprentissage en ligne. Au lieu de suivre systématiquement la séquence $N_1 \rightarrow N_2 \rightarrow N_3$, un sélecteur de type bandit multi-bras observe les gains obtenus par chaque voisinage et apprend progressivement lequel est le plus pertinent selon l'état courant de la recherche.

A. Codage et représentation d'une solution

Comme dans l'AG, une solution est représentée par une permutation $\pi = (\pi_1, \pi_2, \dots, \pi_n)$ des n jobs. La fonction objectif reste le makespan $C_{\max}(\pi)$ calculé à partir des temps de complétion définis dans la formulation mathématique. La solution initiale est construite par l'heuristique NEH [2], qui fournit une permutation de bonne qualité servant de point de départ à la recherche. Ce choix est important sur le plan expérimental : toutes les variantes VNS démarrent depuis la

même solution, ce qui permet d'attribuer les différences de performance au mécanisme de sélection des voisinages plutôt qu'à l'initialisation.

Dans ce cadre, une itération VNS ne modifie jamais la nature de la solution : elle transforme uniquement la permutation courante en une autre permutation valide. La faisabilité est donc préservée par construction, et l'évaluation d'une solution se réduit au calcul de $C_{\max}$.

B. Voisinages utilisés

La VNS exploite trois structures de voisinage complémentaires :

- N_1 : **Swap**, qui échange deux jobs dans la permutation.
- N_2 : **Insertion**, qui retire un job et le réinsère à une autre position.
- N_3 : **2-opt**, qui inverse un segment de la permutation afin de produire une perturbation plus large.

Ces voisinages sont ordonnés du moins perturbateur au plus perturbateur. Le voisinage N_1 favorise l'exploitation locale, tandis que N_2 et N_3 permettent d'explorer des régions plus éloignées de l'espace des permutations. Cette complémentarité est essentielle : un voisinage trop local risque de converger rapidement vers un optimum local, tandis qu'un voisinage trop perturbateur peut ralentir l'intensification. La VNS tire son efficacité de l'alternance contrôlée entre ces deux comportements.

C. Cycle général de la VNS

À chaque itération, la VNS applique trois étapes successives. D'abord, la phase de *shaking* choisit un voisinage N_k et perturbe la meilleure solution courante. Ensuite, une recherche locale complète est appliquée afin d'améliorer la solution générée. Enfin, la phase de *move* accepte la nouvelle permutation si elle diminue le makespan ; sinon, l'algorithme change de voisinage ou poursuit l'exploration.

Dans la VNS classique, le changement de voisinage est déterministe : après un échec dans N_1 , l'algorithme passe à N_2 , puis à N_3 . Cette règle est simple et reproductible, mais elle ne tient pas compte de l'historique des améliorations observées pendant la recherche.

Un mécanisme de redémarrage est également utilisé lorsque la recherche ne progresse plus pendant un nombre fixé d'itérations. La meilleure solution connue est alors perturbée par plusieurs mouvements 2-opt, puis la recherche reprend autour de cette nouvelle permutation. Ce mécanisme évite que l'algorithme reste bloqué dans un bassin d'attraction unique.

D. Composants ML : Sélecteur de Voisinage par Bandit

1) *ML-VNS avec UCB*: La première variante ML-VNS utilise un sélecteur UCB (Upper Confidence Bound) [7], dans lequel chaque voisinage N_k est interprété comme un bras de bandit. Pour chaque bras, le modèle maintient une valeur estimée Q_k et un compteur n_k indiquant le nombre de sélections. Le voisinage choisi à l'itération t maximise le score :

$$Score(k) = Q_k + c \sqrt{\frac{\ln(t+1)}{n_k+1}}, \quad (19)$$

où c contrôle le compromis entre exploitation des voisinages déjà performants et exploration des voisinages encore peu testés.

Après application du voisinage sélectionné, la valeur Q_k est mise à jour à partir de la récompense observée :

$$Q_k \leftarrow (1 - \alpha)Q_k + \alpha R_t, \quad (20)$$

où α est le taux d'apprentissage. Cette mise à jour permet au sélecteur d'identifier progressivement les voisinages les plus rentables.

Dans cette formulation, l'apprentissage est effectué en ligne : il n'exige pas de base d'entraînement préalable. Le modèle apprend directement pendant l'optimisation, à partir des récompenses produites par les mouvements acceptés ou rejetés. Cette propriété est adaptée au PFSP, car la qualité relative des voisinages peut varier selon la taille de l'instance et le niveau de stagnation de la recherche.

2) *Fonction de récompense*: La récompense principale mesure le gain relatif de makespan obtenu après application du voisinage :

$$R_t^{gain} = \frac{\max(0, C_{\max}^{avant} - C_{\max}^{apres})}{C_{\max}^{avant}}. \quad (21)$$

Cependant, lorsque la solution NEH initiale est déjà très compétitive, de nombreux mouvements n'améliorent pas immédiatement le makespan, ce qui peut produire une suite de récompenses nulles. Pour conserver un signal d'apprentissage, nous ajoutons un léger bonus d'exploration :

$$R_t = R_t^{gain} + \frac{\epsilon}{n_k + 1}. \quad (22)$$

Ce terme favorise temporairement les voisinages peu utilisés, puis diminue automatiquement à mesure que leur compteur augmente. Ainsi, même lorsque NEH atteint déjà une solution très compétitive, le sélecteur reçoit un signal minimal qui l'empêche de figer trop tôt sa politique.

3) *Extension par Bandit Contextuel*: Une seconde variante enrichit UCB par un bandit contextuel (Contextual Multi-Armed Bandit, CMAB) [8]. Contrairement à UCB, qui associe un scalaire Q_k à chaque voisinage, le CMAB apprend un vecteur de poids w_k pour chaque bras. À chaque itération, l'état de la recherche est résumé par un vecteur de contexte ϕ_t contenant notamment la taille de l'instance, le niveau de stagnation, le voisinage courant et l'écart au makespan de référence.

Le score contextuel d'un voisinage est alors calculé par :

$$Score(k | \phi_t) = w_k^\top \phi_t + c \sqrt{\frac{\ln(t+1)}{n_k+1}}. \quad (23)$$

Après observation de la récompense, les poids sont ajustés par descente de gradient stochastique :

$$w_k \leftarrow w_k + \eta (R_t - w_k^\top \phi_t) \phi_t, \quad (24)$$

où η est le taux d'apprentissage. Cette extension permet au modèle de changer de stratégie selon le contexte : par exemple,

favoriser l'insertion en phase de stagnation, ou privilégier le swap lors d'une intensification proche d'une bonne solution.

Le CMAB représente donc une extension plus expressive que l'UCB classique : il ne cherche plus seulement le voisinage globalement le plus rentable, mais le voisinage le plus approprié à une situation donnée de la recherche. Cette distinction est particulièrement importante sur les grandes instances, où la recherche alterne plus fréquemment entre phases de progrès, phases de stagnation et phases de diversification.

4) *Rôle du Sélecteur et Conclusion*: Le sélecteur ML joue pour la VNS un rôle analogue à celui de l'agent Q-Learning dans l'AG : il **observe** l'état de la recherche, il **décide** du voisinage à appliquer, il **évalue** l'effet de ce choix via la récompense, puis il **met à jour** sa politique de sélection. La différence principale est que l'action ne porte pas sur des paramètres génétiques, mais sur le choix adaptatif de l'opérateur de voisinage.

Cette hybridation conserve l'interprétabilité de la VNS tout en ajoutant une capacité d'adaptation. Les valeurs Q_k ou les poids contextuels w_k indiquent explicitement quels voisinages sont utiles dans les différentes phases de la recherche, ce qui rend l'approche moins rigide qu'une VNS déterministe et plus transparente qu'un modèle ML entièrement boîte noire.

V. EXPÉRIMENTATIONS, RÉSULTATS ET ANALYSES

Cette section présente séparément les résultats obtenus par les deux hybridations étudiées. La première partie analyse l'approche RL-GA, où l'apprentissage par renforcement règle dynamiquement les paramètres de l'algorithme génétique. La seconde partie analyse l'approche ML-VNS, où l'apprentissage intervient dans la sélection adaptative des voisinages.

A. Résultats de l'Approche RL-GA

1) *Comparaison des performances en makespan*: Le tableau III présente les résultats obtenus sur des instances de grande taille de Taillard [3] ($n \in \{20, 50, 100\}$ jobs et $m = 20$ machines). La colonne HY_moy correspond au makespan moyen de l'approche hybride RL-GA.

TABLE III
COMPARAISON DU MAKESPAN MOYEN : NEH VS AG VS RL-GA

Instance	NEH	AG_moy	HY_moy	Gain/AG
Ta_20x20_i3	2305	2291.0	2287.7	+0.1%
Ta_50x20_i1	3943	3919.7	3887.7	+0.8%
Ta_50x20_i3	4009	3984.3	3941.3	+1.1%
Ta_100x20_i1	6527	6527.0	6486.7	+0.6%
Ta_100x20_i3	6713	6686.0	6660.7	+0.4%

Observation : RL-GA améliore systématiquement l'AG classique sur les instances complexes. Les gains restent modérés, mais ils sont réguliers et deviennent plus pertinents lorsque la taille de l'espace de recherche augmente. Le cas Ta_50x20_i3 illustre clairement cet effet avec un gain de 1.1% par rapport à l'AG statique.

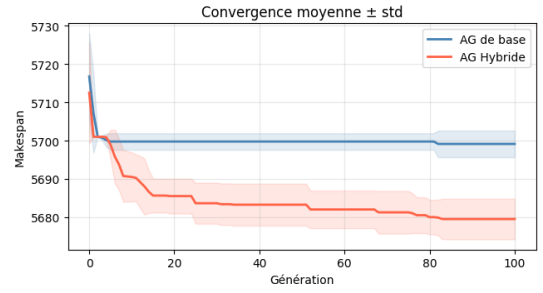


Fig. 2. Comparaison de la convergence entre l'AG standard et l'AG hybride RL-GA.

2) *Précision et stabilité*: Pour évaluer la robustesse de RL-GA, nous utilisons le RPD (Relative Percentage Deviation) par rapport à la solution de référence considérée. Le tableau IV synthétise les performances par classe d'instances.

TABLE IV
SYNTHÈSE DES PERFORMANCES RL-GA PAR CLASSE D'INSTANCES

Classe	RPD_moy	RPD_min	t_moy (s)
Ta_50x5	0.04%	0.00%	10.79
Ta_50x10	0.24%	0.00%	19.18
Ta_100x5	0.01%	0.00%	45.54
Ta_100x10	0.43%	0.00%	84.99

Les résultats montrent une bonne stabilité : le RPD moyen reste inférieur à 0.5% sur les classes considérées, y compris pour les instances de 100 jobs. Le temps de calcul augmente avec la taille des instances, mais reste cohérent avec une métaheuristique destinée à améliorer la qualité finale.

3) *Comportement de l'agent RL*: L'agent RL explore en moyenne 8.5 états sur les 9 états définis, ce qui indique que la représentation basée sur le fitness moyen et l'entropie couvre effectivement les situations rencontrées pendant l'évolution de la population.

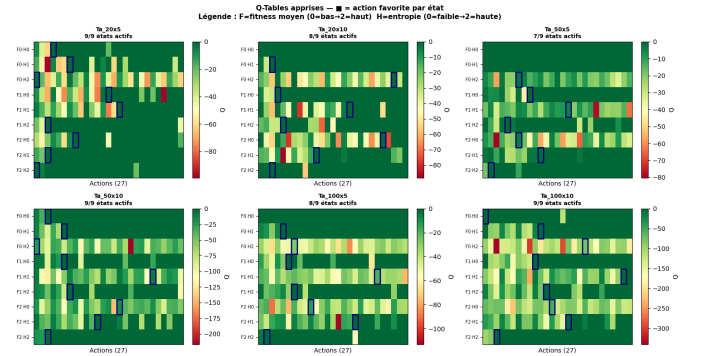


Fig. 3. Visualisation de la Q-Table apprise par l'agent RL-GA.

L'analyse de la Q-Table (Figure 3) montre que l'agent tend à favoriser des taux de mutation élevés lorsque la diversité diminue, ce qui lui permet de réintroduire de l'exploration. À l'inverse, lorsque la population converge vers de bonnes solutions, la politique apprise privilégie des actions plus conservatrices, notamment l'élitisme.

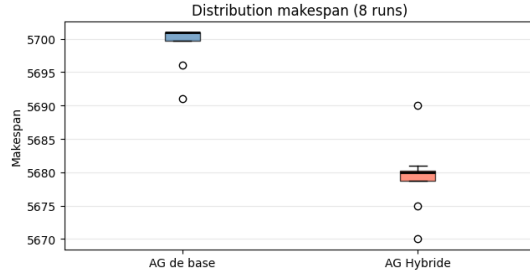


Fig. 4. Distribution de la précision (RPD) pour les différentes classes d'instances.

B. Résultats de l'Approche ML-VNS

1) *Comparaison des variantes VNS*: Le tableau V compare la solution initiale NEH, la VNS classique, la VNS aléatoire, ML-VNS UCB et le bandit contextuel CMAB sur un ensemble d'instances générées de tailles croissantes. Nous retenons ici les résultats qui mettent en évidence une amélioration effective du makespan.

TABLE V
COMPARAISON DES VARIANTES VNS SUR PLUSIEURS INSTANCES

Inst.	NEH	VNS	Alea.	UCB	CMAB
20x5	1350	1341	1341	1341	1341
20x10	1500	1432	1432	1432	1433
50x5	2802	2795	2795	2795	2795
50x10	3264	3094	3114	3107	3094

Observation : les variantes VNS améliorent nettement NEH dès que l'instance devient suffisamment difficile. Sur 20x10, ML-VNS UCB réduit le makespan de 1500 à 1432, soit un gain de 4.53%. Sur 50x10, CMAB égale la VNS classique avec un makespan de 3094 et améliore UCB de 13 unités (3094 contre 3107), ce qui confirme l'intérêt du contexte dans les grandes instances. Il faut également noter que l'objectif du composant ML n'est pas uniquement de battre la VNS classique à chaque instance, mais de fournir un mécanisme de sélection adaptatif et interprétable. Dans les cas où la règle déterministe $N_1 \rightarrow N_2 \rightarrow N_3$ est déjà très efficace, ML-VNS peut égaler la qualité de la VNS tout en produisant des indicateurs supplémentaires sur l'utilité de chaque voisinage.

2) *Analyse de l'apprentissage UCB*: L'évaluation interne de ML-VNS UCB montre que le sélecteur apprend un comportement exploitable lorsque les voisinages produisent des améliorations. Sur l'instance 20x10, le modèle accepte 9 améliorations et obtient un gain final de 68 unités de makespan par rapport à NEH.

TABLE VI
ANALYSE INTERNE DU SÉLECTEUR UCB SUR L'INSTANCE 20x10

Voisinage	Sél.	Acc.	R. moy.	R. max
N_1 Swap	100	4.0%	0.00097	0.04720
N_2 Insertion	105	3.8%	0.00059	0.01049
N_3 2-opt	95	1.1%	0.00055	0.01000

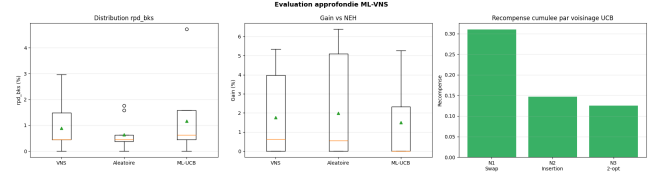


Fig. 5. Analyse du sélecteur UCB : sélections, améliorations acceptées et récompenses cumulées par voisinage.

La Figure 5 complète le tableau en montrant la répartition des choix et des récompenses. Le voisinage N_1 présente la meilleure récompense moyenne sur l'instance 20x10, ce qui indique que les mouvements de type swap jouent un rôle important au début de la recherche. Les insertions contribuent également aux améliorations acceptées, tandis que le 2-opt sert davantage à maintenir la diversification.

Cette lecture est cohérente avec le rôle attendu des voisinages. Le swap permet des corrections fines et peu coûteuses de l'ordre des jobs ; l'insertion modifie plus fortement la position relative d'un job et peut corriger des blocs mal placés ; le 2-opt agit comme une perturbation plus large, utile lorsque la recherche doit quitter une zone stagnante. Le sélecteur UCB rend cette répartition observable à travers les compteurs, les taux d'acceptation et les récompenses.

3) *Apport du bandit contextuel*: Le tableau VII met l'accent sur la comparaison entre UCB et CMAB, car elle mesure directement l'intérêt d'utiliser le contexte de la recherche dans la sélection du voisinage.

TABLE VII
APPORT DU CONTEXTUAL MAB PAR RAPPORT À ML-VNS UCB

Instance	UCB	CMAB	Gain CMAB/UCB
20x5	1341	1341	0
20x10	1432	1433	-1
50x5	2795	2795	0
50x10	3107	3094	+13

Sur les instances les plus grandes, le bandit contextuel devient plus pertinent : il égale la meilleure VNS sur 50x5 et 50x10, et sur 50x10 il surpasse clairement UCB. Ce résultat suggère que l'information contextuelle (stagnation, taille de l'instance, écart à la référence et voisinage courant) aide le modèle à mieux décider quand intensifier ou diversifier la recherche. En revanche, le coût de calcul du CMAB est plus élevé, ce qui constitue un compromis temps/qualité à prendre en compte.

L'intérêt du CMAB est donc principalement qualitatif et stratégique : il montre que la sélection de voisinage peut être conditionnée par l'état de la recherche plutôt que par une moyenne globale. Sur 50x10, cette information contextuelle permet de retrouver le meilleur makespan observé par la VNS classique, tout en corrigeant la faiblesse de l'UCB stationnaire sur cette instance. Ce résultat soutient l'hypothèse selon laquelle les modèles contextuels deviennent plus utiles lorsque l'espace de recherche est plus vaste et plus hétérogène.

4) *Synthèse VNS*: Les résultats VNS confirment trois points. Premièrement, NEH fournit une base de très bonne qualité, mais les variantes VNS apportent des gains importants sur les instances moyennes et grandes. Deuxièmement, ML-VNS UCB n'est pas toujours dominant face à une VNS classique bien réglée, mais il fournit un signal d'apprentissage mesurable et interprétable. Troisièmement, CMAB apporte une amélioration ciblée sur les instances complexes, notamment sur 50x10 où il réduit le makespan de 13 unités par rapport à UCB tout en restant transparent grâce aux poids contextuels appris.

En synthèse, l'approche ML-VNS ne remplace pas systématiquement la VNS classique, mais elle enrichit la métaheuristique par une capacité d'auto-adaptation. Elle est particulièrement pertinente lorsque l'ordre optimal des voisinages n'est pas connu a priori, ou lorsque les instances présentent des comportements de stagnation variables. Pour une validation plus robuste, une évaluation future devrait utiliser un plus grand nombre de runs, un budget CPU strictement identique et l'ensemble complet des instances de Taillard [3] avec RPD calculé par rapport aux meilleures solutions connues.

VI. CONCLUSION ET PERSPECTIVES

Ce travail a étudié l'apport de l'apprentissage automatique dans la résolution du Permutation Flow Shop Problem, un problème d'ordonnancement NP-difficile où l'objectif principal est la minimisation du makespan. Deux hybridations ont été proposées et analysées : une approche RL-GA, dans laquelle un agent Q-Learning pilote dynamiquement les paramètres d'un algorithme génétique, et une approche ML-VNS, dans laquelle un sélecteur de voisinage apprend à orienter la recherche locale.

Les résultats obtenus montrent d'abord que l'approche RL-GA améliore de manière régulière l'algorithme génétique classique sur les instances de grande taille. L'agent RL exploite l'information issue du fitness moyen et de l'entropie de population pour adapter les taux de sélection et de mutation. Cette adaptation permet de mieux équilibrer exploration et exploitation : lorsque la diversité diminue, l'agent favorise des actions plus exploratoires ; lorsque la population converge, il privilégie des choix plus conservateurs afin de stabiliser les bonnes solutions.

La seconde contribution concerne l'approche ML-VNS. Les expérimentations confirment que la VNS constitue une base très efficace pour le PFSP, en particulier lorsque la solution NEH initiale peut encore être améliorée. Le sélecteur UCB apporte un mécanisme interprétable de sélection adaptative des voisinages, tandis que le bandit contextuel CMAB enrichit cette logique en tenant compte de l'état courant de la recherche. Sur les instances les plus complexes, notamment 50x10, le CMAB parvient à égaler la meilleure VNS observée et à améliorer l'UCB stationnaire, ce qui confirme l'intérêt de l'information contextuelle pour guider l'intensification et la diversification.

Ces résultats doivent toutefois être interprétés avec prudence. D'une part, la VNS classique reste très compétitive

lorsque l'ordre déterministe des voisinages est déjà bien adapté au problème. D'autre part, les variantes apprenantes introduisent un coût de calcul supplémentaire, particulièrement visible pour le bandit contextuel. L'intérêt du ML ne réside donc pas uniquement dans une amélioration systématique du makespan, mais aussi dans sa capacité à rendre la recherche plus adaptative, plus explicable et plus robuste face à des configurations où le meilleur voisinage n'est pas connu à l'avance.

Plusieurs perspectives peuvent prolonger ce travail. Premièrement, il serait nécessaire d'étendre l'évaluation à l'ensemble complet des instances de Taillard, avec un nombre de runs plus élevé et un budget CPU strictement identique pour chaque méthode. Deuxièmement, les hyperparamètres des agents apprenants, notamment α , γ , ϵ , c et η , pourraient être calibrés plus systématiquement par recherche sur grille ou par optimisation bayésienne. Troisièmement, le modèle contextuel pourrait être enrichi par des approches plus expressives, telles que LinUCB, NeuralUCB ou un réseau de neurones léger, afin de capturer des interactions non linéaires entre la stagnation, la taille de l'instance et le type de voisinage. Enfin, une perspective importante consiste à étudier le transfert inter-instances : une politique apprise sur un ensemble d'instances pourrait être réutilisée comme point de départ sur de nouvelles instances, réduisant ainsi le coût d'apprentissage en ligne.

En conclusion, l'hybridation ML-métaheuristique apparaît comme une voie prometteuse pour l'optimisation du flow shop. Elle ne remplace pas les métaheuristiques classiques, mais les complète en leur ajoutant une capacité d'adaptation automatique. Les résultats obtenus indiquent que cette combinaison est particulièrement pertinente pour les instances complexes, où l'équilibre entre diversification, intensification et interprétabilité conditionne directement la qualité des solutions produites.

REMERCIEMENTS

Les auteurs remercient M. Kechid Amine et Mme Bessedik Malika pour leur encadrement tout au long de ce travail.

RÉFÉRENCES

- [1] M. R. Garey, D. S. Johnson, and R. Sethi, "The complexity of flowshop and jobshop scheduling," *Mathematics of Operations Research*, vol. 1, no. 2, pp. 117–129, 1976.
- [2] M. Nawaz, E. E. Enscore, Jr., and I. Ham, "A heuristic algorithm for the m-machine, n-job flow-shop sequencing problem," *Omega*, vol. 11, no. 1, pp. 91–95, 1983.
- [3] E. Taillard, "Benchmarks for basic scheduling problems," *European Journal of Operational Research*, vol. 64, no. 2, pp. 278–285, 1993.
- [4] N. Mladenović and P. Hansen, "Variable neighborhood search," *Computers & Operations Research*, vol. 24, no. 11, pp. 1097–1100, 1997.
- [5] P. Hansen, N. Mladenović, and J. A. Moreno Pérez, "Variable neighbourhood search: methods and applications," *Annals of Operations Research*, vol. 175, no. 1, pp. 367–407, 2010.
- [6] R. Ruiz and T. Stützle, "A simple and effective iterated greedy algorithm for the permutation flowshop scheduling problem," *European Journal of Operational Research*, vol. 177, no. 3, pp. 2033–2049, 2007.
- [7] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," *Machine Learning*, vol. 47, no. 2–3, pp. 235–256, 2002.
- [8] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A contextual-bandit approach to personalized news article recommendation," in *Proc. 19th International Conference on World Wide Web*, 2010, pp. 661–670.